Sys-ID Concatenate + Non-convex Landscape Q&A

Verbatim Q&A from 2026-05-08 session

Q1

"Concatenate all params to optimize into one vector, then run gradient descent on this vector with the loss function?"

A1: Exactly right - but with some nuance worth clarifying

Mathematically you are correct: concatenate + gradient descent

```
theta = [knife_pos_x, knife_pos_y, knife_pos_z, stiffness]   # 4-dim vector
loss = MSE(sim(theta), target)
grad = dloss/dtheta = [dloss/dk_x, dloss/dk_y, dloss/dk_z, dloss/dstiff]   # 4-dim
theta_new = theta - lr * grad   # standard gradient descent
```

This is the essence of multi-param sys-ID.

But in implementation, we do NOT actually concatenate (only mathematically equivalent)

In code we keep them as two independent wp.arrays:

```
knife_pos_arr = wp.array([wp.vec3(...)], dtype=wp.vec3, requires_grad=True)
stiff_arr = wp.array([0.5], dtype=wp.float32, requires_grad=True)
```

Tape backward gives both at once:

```
grad_kp = knife_pos_arr.grad   # (1, 3) vec3
grad_st = stiff_arr.grad       # (1,) float
```

Why NOT actually concatenate? Because parameter scales differ enormously:

Param	Unit	Typical value	Typical grad magnitude
knife_pos	meter	~0.5	~1e-3
stiffness	dimensionless	~0.5	~1e-5

If you just concatenate to [k_x, k_y, k_z, stiff] with single LR:

- LR too big: knife_pos updates are reasonable, stiffness updates too aggressive

- LR too small: stiffness updates reasonable, knife_pos converges slowly

Adam intelligence: per-dim adaptive

Adam gives EACH dim an independent effective LR (based on RMS of dloss/dx history). So even if concatenated, Adam adapts per-dim. EQUIVALENT to a diagonal preconditioner.

In our code we used 2 independent Adam instances (knife with LR=0.005, stiff with LR=0.02) - mathematically little difference from 1 4-dim Adam, just that the OUTER LR base is manually tuned per param.

The real difficulty is NOT concatenate, it is non-convex coupling

Sys-ID difficulty:

```
loss(knife_pos, stiff) is a coupled function of these two
  - Soft stiffness + far knife = similar loss to hard stiffness + near knife
  - Multiple (knife, stiff) combinations may give same target -> multiple local minima
```

Step 5 v2 failure was exactly this - Adam found a local min (stiff=0.05, knife off) loss=4.1e-4, less than INIT 5.4e-4 but greater than GT 0. Gradient was locally correct, but globally Adam got stuck in wrong basin.

Standard methods to handle non-convexity

Method	Effort	Effect
Closer INIT (start near GT)	Easy	Avoid local minima - Step 5 v3 does this
Multi-restart (10 random INITs, take best)	Medium	Brute-force find global
L-BFGS + line search (adaptive step)	Medium	More robust than Adam
Trust region / Levenberg-Marquardt	Hard	Classic non-linear least squares
Bayesian Optimization / TPE	Hard	Black-box global, no gradient needed

In real projects: use Adam for local refinement, but FIND A GOOD INIT FIRST (e.g., real calibration starts from reasonable prior).

One-line summary

OK: "Concatenate + gradient descent" is correct. Adam handles per-dim scale automatically.

WARN: But sys-ID difficulty is NOT in concatenate, it is in non-convex loss landscape. INIT too far from GT -> falls into local minima. Step 5 v3 with closer INIT converges.

Q2

"I do not quite understand: WARN: But sys-ID difficulty is not in concatenate, it is in non-convex loss landscape. INIT too far from GT falls in local minima. Step 5 v3 with closer INIT converges. What does this mean, why did our previous Step 5 not work?"

A2: Explain with concrete numbers + diagrams

What does "loss landscape non-convex" mean

Loss = MSE(sim_pos, target_pos). Loss is a 4-dim function (4 parameters), but PLOTTING is only 2-dim. Simplified: look at loss along the stiffness axis only:

Convex function (ideal case)

```
loss
  ^
  | \                          /
  |  \                        /
  |   \                      /
  |    \                    /
  |     \__________________/
  |               *                  <- GT (unique lowest point)
  +------------------------------> stiffness
       0.0       0.85         1.0
```

From anywhere, gradient descent reaches GT. Adam ALWAYS WINS.

Non-convex (our actual sim)

```
loss
  ^
  | \                          /
  |  \                        /
  |   \       ___            /
  |    \     /   \           /
  |     \___/     \_________/
  |      ^          ^
  |  local min    GT min
  +------------------------------> stiffness
       0.05      0.85         1.0
```

Two minima:

- GT min: stiffness=0.85, loss=0

- Local min: stiffness=0.05 ("mush" state), loss=4e-4 (lower than INIT, higher than GT)

Gradient descent only sees LOCAL slope, not global:

- INIT on GT min side -> walks to GT

- INIT on local min side -> walks to local min (NEVER reaches GT)

What actually happened to us

v1/v2: INIT (0.7, 0.55, 0.15, 0.5) - far from GT

```
Where is INIT (stiff=0.5)?
  loss(0.5) ~ 5.4e-4
  Higher than local min (stiff=0.05, loss=4e-4)
```

```
  But Adam looks at "where does loss go down fastest" -> walks toward decreasing
  stiff (steeper slope on local-min side)

Trajectory: 0.5 -> 0.45 -> 0.30 -> 0.13 -> 0.05 (clamp) -> stuck
Final: stiffness=0.05, knife also wandering, loss=4e-4
```

Adam DID converge - but to the wrong place.

v3: INIT (0.83, 0.51, 0.13, 0.78) - close to GT

```
Where is INIT (stiff=0.78)?
  loss(0.78) ~ 7e-5  <- already close to GT loss
  Adam sees: walking toward increasing stiff drops loss fastest -> moves to GT

Trajectory: 0.78 -> 0.80 -> 0.82 -> 0.85 -> 0.85 (over-shoot) -> 0.85 (settle)
Final: stiffness=0.8502, knife exact GT, loss=6e-12
```

Adam went the RIGHT direction this time.

Why physically there is a local min

Intuition: when matching sim to target, MULTIPLE (knife, stiff) combos may give similar output:

Param combination	Behavior	Diff vs target
GT: stiff=0.85, knife at (0.85, 0.5, 0.12)	Hard mesh, knife pokes edge	Perfect match (=target)
Local min: stiff=0.05, knife somewhere	Soft mesh, knife irrelevant (mesh collapses by itself)	Partial match (some surface pts align)

Both sim behaviors give different pos patterns, but partial matches with target are both BETTER than INIT. Adam finds the closest "better than INIT" basin, may not be GT basin.

How real sys-ID avoids this

Standard ML practice (also what SliceIt uses)

Two-stage optimization:

1. TPE / Bayesian Optimization (zeroth-order, slow but global): random sampling to find which basin is GT basin -> gives a reasonable INIT

2. Adam / L-BFGS (gradient-based, fast but local): from reasonable INIT, refine to machine precision

```
Stage 1 (TPE 50 trials):  random INIT -> find lowest loss INIT -> e.g. (0.83, 0.51, 0.13, 0.78)
Stage 2 (Adam 100 iter):  from stage 1 INIT -> refine to GT (0.85, 0.5, 0.12, 0.85)
```

This is the "Optuna -> Adam two-stage" pattern in SliceIt paper - not gimmicky, it is the STANDARD operating procedure for non-convex sys-ID.

One-line answer to you

Step 5 v1/v2 failure was NOT a sys-ID framework bug, it was INIT falling in the wrong basin. Step 5 v3 INIT in GT basin -> Adam went directly to GT.

Real phantom calibration: first use TPE / random search to find a reasonable INIT (guaranteed near GT basin), then Adam for local refinement.